

```
import pandas as pd
import numpy as np
import scipy.stats as sps
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
```

```
from google.colab import files
```

```
df = files.upload()
```

Выбрать файлы

Файл не выбран

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving random_fitness_dataset.csv to random_fitness_dataset.csv

```
df = pd.read_csv('random_fitness_dataset.csv')
df
```

	record_id	timestamp	user_id	age	gender	activity_type	duration_min	calories_burned	heart_rate_avg	steps	sleep_h
0	1	2025-07-22 09:22:54	U026	67	Male	Cycling	7	322	175	3591	
1	2	2025-05-28 13:53:54	U039	16	Female	Strength Training	83	265	113	22312	
2	3	2025-08-07 01:36:54	U046	62	Female	Cycling	118	508	98	5418	
3	4	2025-07-17 01:56:54	U048	56	Female	Swimming	75	270	150	19809	
4	5	2025-08-11 03:22:54	U042	42	Other	Yoga	76	308	141	12562	
...	...	...	...	...	...	...	...	...	...	...	...

```
df.shape
```

(200, 12)

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   record_id              200 non-null   int64
1   timestamp              200 non-null   object
2   user_id                200 non-null   object
3   age                    200 non-null   int64
4   gender                 200 non-null   object
5   activity_type          200 non-null   object
6   duration_min           200 non-null   int64
7   calories_burned        200 non-null   int64
8   heart_rate_avg         200 non-null   int64
9   steps                  200 non-null   int64
10  sleep_hours             200 non-null   float64
11  weight_kg               200 non-null   float64
dtypes: float64(2), int64(6), object(4)
memory usage: 18.9+ KB
```

```
df.describe()
```

	record_id	age	duration_min	calories_burned	heart_rate_avg	steps	sleep_hours	weight_kg
count	200.000000	200.000000	200.000000	200.000000	200.000000	200.000000	200.000000	200.000000
mean	100.500000	44.380000	64.025000	295.765000	120.935000	14020.82000	6.313000	70.152000
std	57.879185	15.417982	34.693987	120.563834	34.100366	6428.05981	1.911518	14.129562
min	1.000000	16.000000	5.000000	50.000000	60.000000	370.00000	3.000000	35.300000
25%	50.750000	33.000000	33.000000	220.000000	89.750000	9585.00000	4.775000	61.450000
50%	100.500000	43.000000	68.000000	311.500000	121.000000	14857.00000	6.500000	70.600000
75%	150.250000	58.000000	95.000000	378.000000	150.250000	19007.75000	7.900000	80.100000
max	200.000000	69.000000	119.000000	585.000000	179.000000	24984.00000	9.500000	105.200000

df.describe(include='object')

	timestamp	user_id	gender	activity_type
count	200	200	200	200
unique	199	49	3	7
top	2025-07-08 21:56:54	U030	Female	Yoga
freq	2	9	69	42

df.describe(include='all')

	record_id	timestamp	user_id	age	gender	activity_type	duration_min	calories_burned	heart_rate_avg	
count	200.000000	200	200	200.000000	200	200	200.000000	200.000000	200.000000	2
unique	NaN	199	49	NaN	3	7	NaN	NaN	NaN	
top	NaN	2025-07-08 21:56:54	U030	NaN	Female	Yoga	NaN	NaN	NaN	
freq	NaN	2	9	NaN	69	42	NaN	NaN	NaN	
mean	100.500000	NaN	NaN	44.380000	NaN	NaN	64.025000	295.765000	120.935000	140
std	57.879185	NaN	NaN	15.417982	NaN	NaN	34.693987	120.563834	34.100366	64
min	1.000000	NaN	NaN	16.000000	NaN	NaN	5.000000	50.000000	60.000000	3
25%	50.750000	NaN	NaN	33.000000	NaN	NaN	33.000000	220.000000	89.750000	95
50%	100.500000	NaN	NaN	43.000000	NaN	NaN	68.000000	311.500000	121.000000	148
75%	150.250000	NaN	NaN	58.000000	NaN	NaN	95.000000	378.000000	150.250000	190

df.duplicated().sum()

np.int64(0)

df.isna().sum()

	0
record_id	0
timestamp	0
user_id	0
age	0
gender	0
activity_type	0
duration_min	0
calories_burned	0
heart_rate_avg	0
steps	0
sleep_hours	0
weight_kg	0

dtype: int64

Описание датафрейма по его основным параметрам

Размерность датафрейма: 200 строк и 12 столбцов

Пропуски в данных отсутствуют

Дубликаты отсутствуют

Количественных признаков 8 (6 целочисленных и 2 вещественных)

Качественных признаков 4

Выполним необходимые преобразования

Заменим название первого столбца record_id на serial_number (порядковый номер)

```
df = df.rename(columns={'record_id': 'serial_numder'})
df
```

	serial_numder	timestamp	user_id	age	gender	activity_type	duration_min	calories_burned	heart_rate_avg	steps	sleep
0	1	2025-07-22 09:22:54	U026	67	Male	Cycling	7	322	175	3591	1
1	2	2025-05-28 13:53:54	U039	16	Female	Strength Training	83	265	113	22312	1
2	3	2025-08-07 01:36:54	U046	62	Female	Cycling	118	508	98	5418	1
3	4	2025-07-17 01:56:54	U048	56	Female	Swimming	75	270	150	19809	1
4	5	2025-08-11 03:22:54	U042	42	Other	Yoga	76	308	141	12562	1
...	...	...	...	...	...	...	...	...	...	...	...

Дубликатов и пропусков в датафрейме нет

Анализ качественных признаков

1. timestamp

```
df['timestamp'].describe()
```

timestamp	
count	200
unique	199
top	2025-07-08 21:56:54
freq	2

dtype: object

```
df['timestamp'].value_counts()
```

timestamp	count
2025-07-08 21:56:54	2
2025-05-28 13:53:54	1
2025-07-22 09:22:54	1
2025-07-17 01:56:54	1
2025-08-11 03:22:54	1
...	...
2025-05-28 13:00:54	1
2025-10-10 22:31:54	1
2025-06-12 04:53:54	1
2025-10-11 21:37:54	1
2025-07-11 04:47:54	1

199 rows × 1 columns

dtype: int64

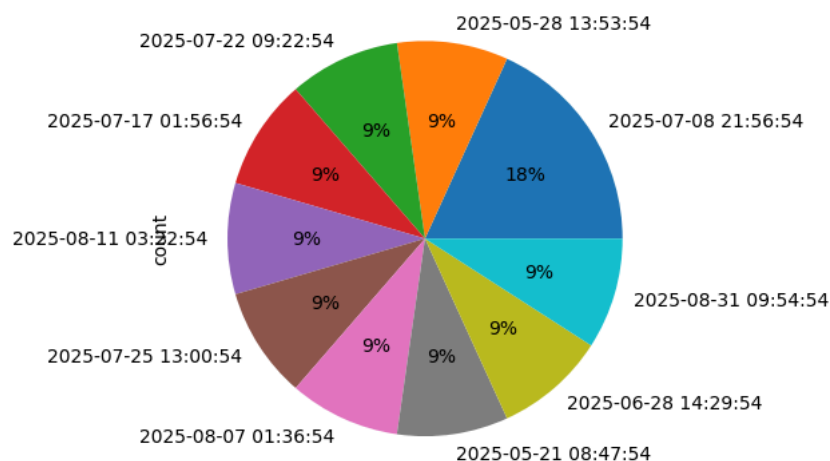
df['timestamp'].value_counts(normalize=True)

timestamp	proportion
2025-07-08 21:56:54	0.010
2025-05-28 13:53:54	0.005
2025-07-22 09:22:54	0.005
2025-07-17 01:56:54	0.005
2025-08-11 03:22:54	0.005
...	...
2025-05-28 13:00:54	0.005
2025-10-10 22:31:54	0.005
2025-06-12 04:53:54	0.005
2025-10-11 21:37:54	0.005
2025-07-11 04:47:54	0.005

199 rows × 1 columns

dtype: float64

```
df['timestamp'].value_counts().head(10).plot.pie(autopct='%0.0f%%', fontsize = 10)
plt.show()
```



Выводы

Распределение временных меток по уникальности - подавляющее большинство временных меток уникальны, составляя 99.5% от всех записей (199 из 200), тогда как повторяющаяся метка встречается всего в 0.5% случаев (1 из 200). Наименее распространенный случай - дублирование временных меток, представленное лишь одним временным значением 2025-07-08 21:56:54, которое встречается дважды. Наиболее распространенный сценарий - уникальные временные метки, охватывающие равномерно период с мая по октябрь 2025 года.

2. user_id

```
df['user_id'].describe()
```

	user_id
count	200
unique	49
top	U030
freq	9

dtype: object

```
df['user_id'].value_counts()
```

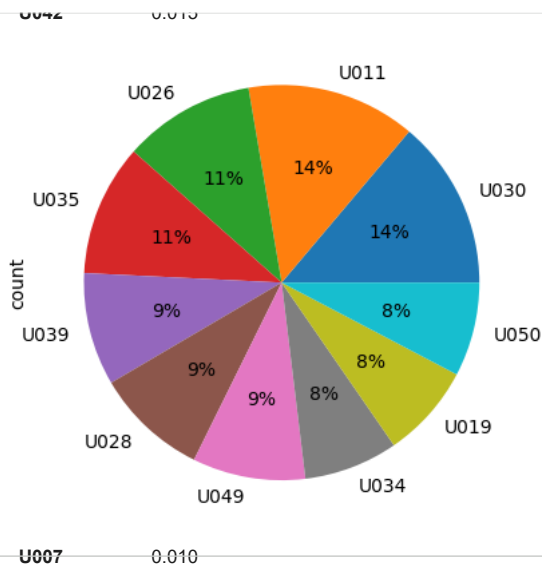
	count
U030	9
U011	9
U026	7
U035	7
U039	6
U028	6
U049	6
U034	5
U019	5
U050	5
U038	5
U025	5
U023	5
U008	5
U013	5
U041	5
U021	5
U004	4
U022	4
U001	4
U033	4
U014	4
U015	4
U031	4
U046	4
U005	4
U045	4
U024	4
U018	4
U002	4

```
df['user_id'].value_counts(normalize=True)
```

U042	3
------	---

U048	prop
U029	3
U030	0.045
U047	3
U026	0.035
U044	3
U039	0.030
U028	3
U049	0.030
U036	3
U019	0.025
U050	2
U038	0.025
U025	2
U023	0.025
U009	1
U013	0.025
U041	0.025
U021	0.025
U004	0.020
U022	0.020
U001	0.020
U033	0.020
U014	0.020
U015	0.020
U031	0.020
U046	0.020
U005	0.020
U045	0.020
U024	0.020
U018	0.020
U002	0.020

```
df['user_id'].value_counts().head(10).plot.pie(autopct='%0.0f%%', fontsize = 10)
plt.show()
```



Вывод: U037 0.010

Распределение пользователей по активности - данные охватывают 49 уникальных пользователей, при этом активность распределена неравномерно. Наиболее активные пользователи U030 и U011 составляют по 4.5% от всех записей (по 9 записей

каждый). Далее следуют U026 и U035 - по 3.5% (по 7 записей). Около 65% пользователей имеют от 3 до 9 записей, что указывает на регулярное использование фитнес-трекера. При этом 6 пользователей (12%) представлены всего одной записью, что может свидетельствовать о разовой активности или новичках в системе.

3. gender

```
df['gender'].describe()
```

gender	
count	200
unique	3
top	Female
freq	69

dtype: object

```
df['gender'].value_counts()
```

count	
gender	
Female	69
Other	68
Male	63

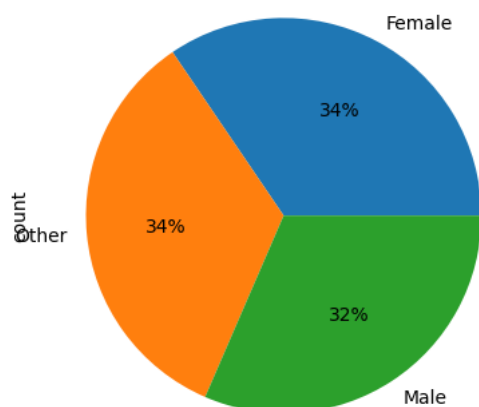
dtype: int64

```
df['gender'].value_counts(normalize=True)
```

proportion	
gender	
Female	0.345
Other	0.340
Male	0.315

dtype: float64

```
df['gender'].value_counts().head(10).plot.pie(autopct='%0f%%', fontsize = 10)  
plt.show()
```



Выводы

Гендерное распределение практически равномерное - все три категории представлены примерно одинаково. Женщины (Female) составляют наибольшую долю - 34.5% (69 записей), категория "Other" следует с 34% (68 записей), мужчины (Male) представлены 31.5% (63 записи). Разница между наиболее и наименее представленными группами составляет всего 3%, что

указывает на сбалансированность выборки по гендерному признаку и обеспечивает репрезентативность для сравнительного анализа активности разных групп.

4. activity_type

```
df['activity_type'].describe()
```

activity_type	
count	200
unique	7
top	Yoga
freq	42

dtype: object

```
df['activity_type'].value_counts()
```

count	
activity_type	
Yoga	42
Walking	30
Strength Training	27
HIIT	27
Swimming	27
Running	24
Cycling	23

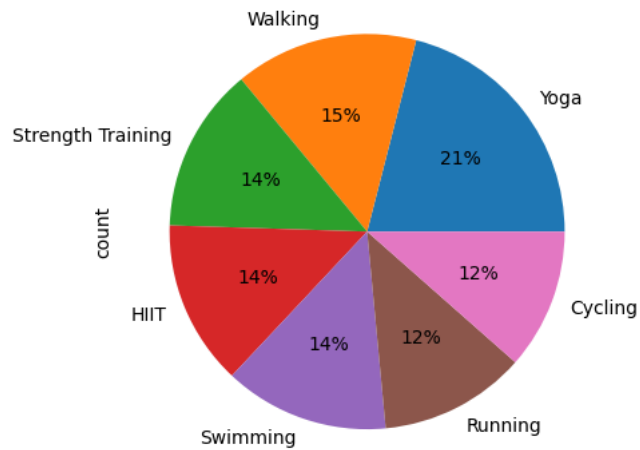
dtype: int64

```
df['activity_type'].value_counts(normalize=True)
```

proportion	
activity_type	
Yoga	0.210
Walking	0.150
Strength Training	0.135
HIIT	0.135
Swimming	0.135
Running	0.120
Cycling	0.115

dtype: float64

```
df['activity_type'].value_counts().head(10).plot.pie(autopct='%0f%%', fontsize = 10)  
plt.show()
```

Выводы

Йога – наиболее популярный вид активности, составляя 21% от всех тренировок (42 записи). Далее следуют ходьба (15%), силовые тренировки (13.5%), HIIT (13.5%) и плавание (13.5%) с практически одинаковой популярностью. Бег (12%) и велоспорт (11.5%) завершают список, образуя относительно равномерное распределение между семью видами активностей. Разница между самым популярным (йога) и наименее популярным (велоспорт) видом активности составляет 9.5%, что указывает на разнообразии предпочтений пользователей без явного доминирования одного типа тренировок.

Анализ количественных признаков

1. age

```
df['age'].describe()
```

	age
count	200.000000
mean	44.380000
std	15.417982
min	16.000000
25%	33.000000
50%	43.000000
75%	58.000000
max	69.000000

dtype: float64

```
df['age'].value_counts().head(10)
```

```
count
age
42    13
66     9
34     8
23     7
31     6
68     6
39     6
60     6
37     6
58     6
```

dtype: int64

```
df['age'].value_counts().head(10)
```

```
count
age
42    13
66     9
34     8
23     7
31     6
68     6
39     6
60     6
37     6
58     6
```

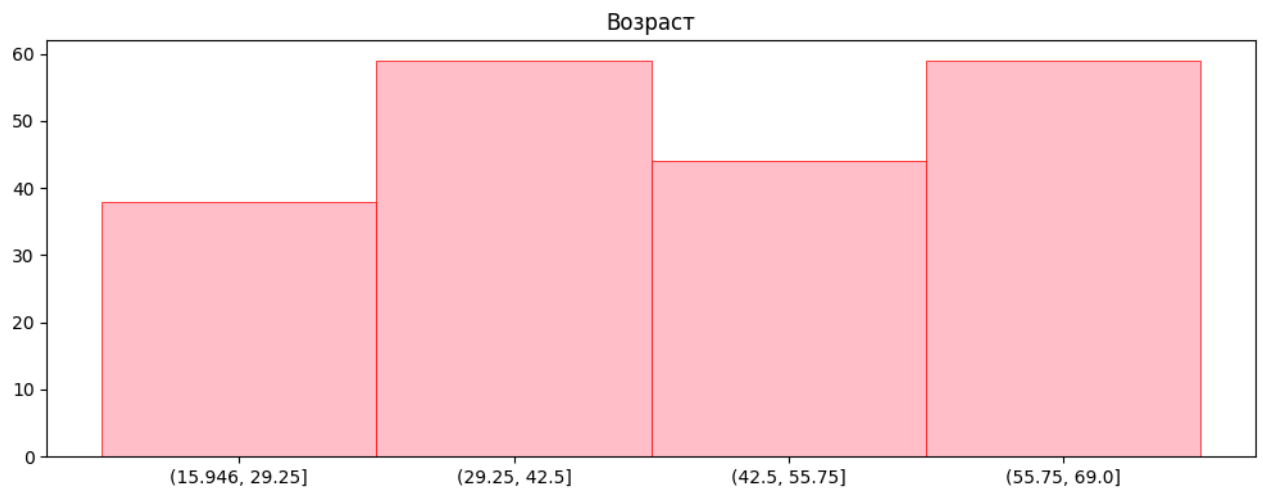
dtype: int64

```
df['age'].value_counts(bins=4).sort_index()
```

```
count
(15.946, 29.25]    38
(29.25, 42.5]     59
(42.5, 55.75]     44
(55.75, 69.0]     59
```

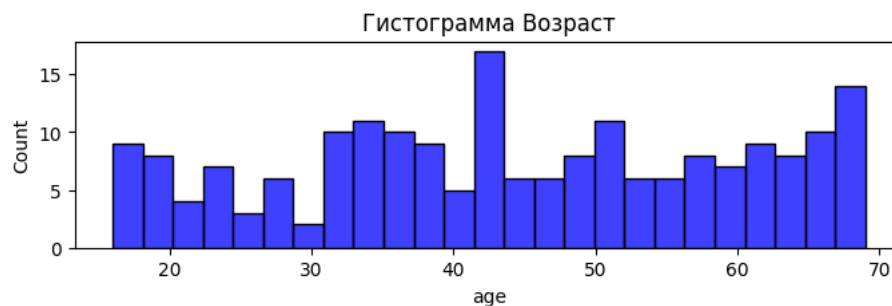
dtype: int64

```
intervals = ['(15.946, 29.25]', '(29.25, 42.5]', '(42.5, 55.75]', '(55.75, 69.0]']
frequencies = [38, 59, 44, 59]
fig, ax = plt.subplots(figsize=[10, 4])
x_pos = range(len(intervals))
plt.bar(x_pos, frequencies, color='pink', width=1.0, edgecolor='red', linewidth=0.5)
plt.xticks(x_pos, intervals)
plt.title('Возраст')
plt.tight_layout()
plt.show()
```



```
fig, ax = plt.subplots(figsize=[8, 2])
sns.histplot(data=df['age'], bins=25, color='blue')
ax.set_title('Гистограмма Возраст', fontsize = 12)

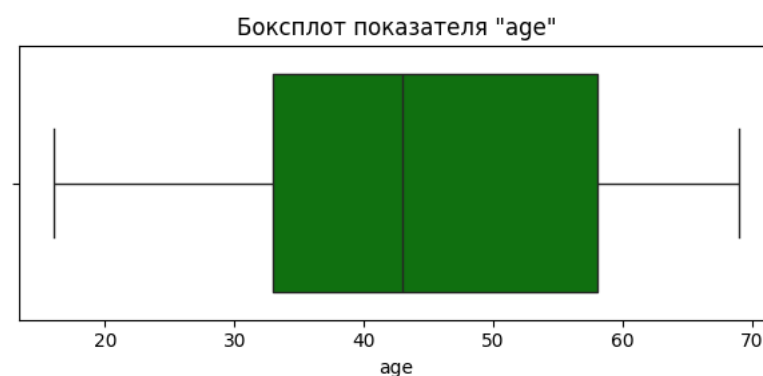
plt.show()
```



```
fig, ax = plt.subplots(1, 1, figsize=[6, 3])

sns.boxplot(data=df, x='age', color='green', ax=ax)

plt.title('Боксплот показателя "age"')
plt.tight_layout()
plt.show()
```

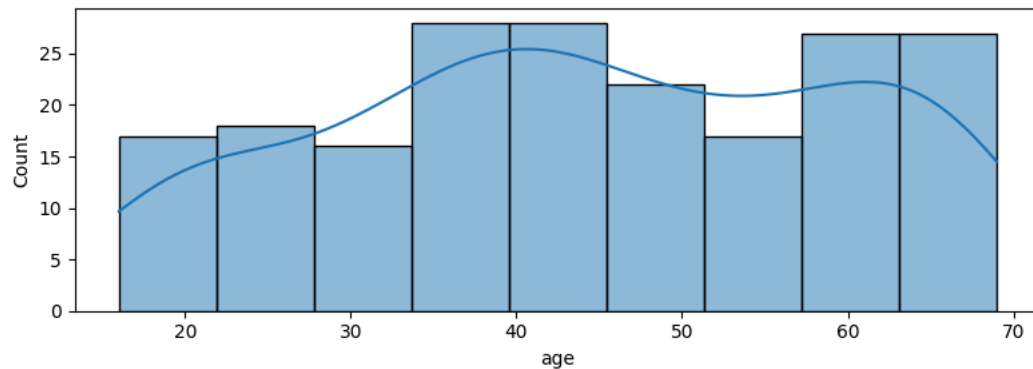


```
print('среднее = {:.3f}'.format(df.age.mean()))
print('медиана = {:.3f}'.format(df.age.median()))
print('мода = {}'.format(df.age.mode()[0]))
print('асимметрия = {:.3f}'.format(sps.skew(df.age)))
print('эксцесс = {:.3f}'.format(sps.kurtosis(df.age)))
```

```
среднее = 44.380
медиана = 43.000
мода = 42
асимметрия = -0.110
эксцесс = -1.076
```

```
# Гистограмма и функция ядерной плотности (kde)
plt.figure(figsize=[8,3])
```

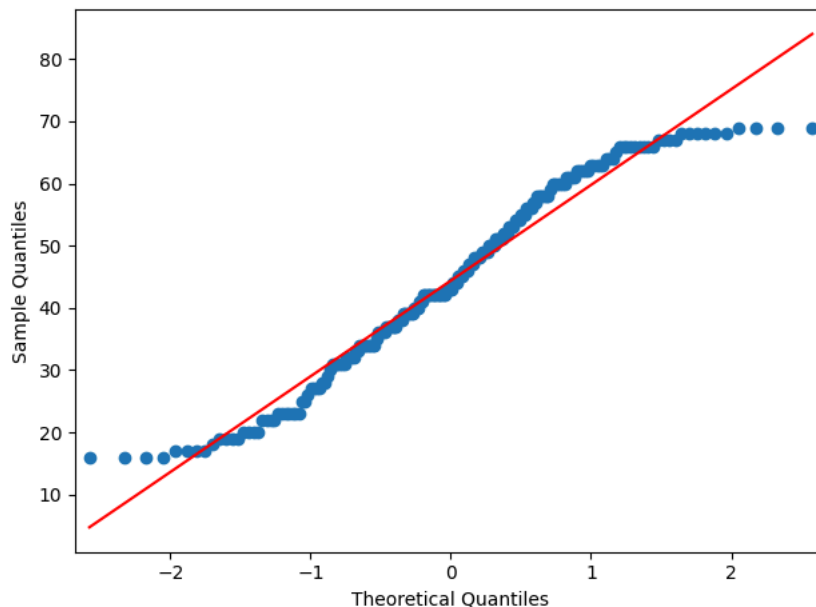
```
sns.histplot(data=df, x='age', kde=True)
plt.tight_layout()
plt.show()
```



```
# Подключить функцию qqplot
from statsmodels.graphics.gofplots import qqplot

# Квантильная диаграмма Q-Q
plt.figure(figsize=[6,4])
qqplot(df.age, line='s')
plt.tight_layout()
plt.show()
```

<Figure size 600x400 with 0 Axes>



Среднее арифметическое, медиана и мода близки по значению. Асимметрия и эксцесс близки к нулю. Можно считать распределение признака возраста близким к нормальному.

```
# Критерий согласия Шапиро-Уилка
sps.shapiro(df.age)
```

```
ShapiroResult(statistic=np.float64(0.9566336800320712), pvalue=np.float64(8.696726313849002e-06))
```

Вывод

Значения варьируются от 16 до 69

Среднее значение: 44.38

Медиана: 43 (50% пользователей имеют возраст 43 и ниже)

Первый квартиль: 33 (25% пользователей имеют возраст 33 и ниже)

Третий квартиль: 58 (75% пользователей имеют возраст 58 и ниже)

Стандартное отклонение от среднего значения: 15.42

Наибольшее количество пользователей находятся в возрасте от 29.25 до 42.5 или от 55.75 до 69

2. duration_min

```
df['duration_min'].describe()
```

	duration_min
count	200.000000
mean	64.025000
std	34.693987
min	5.000000
25%	33.000000
50%	68.000000
75%	95.000000
max	119.000000

dtype: float64

```
df['duration_min'].value_counts()
```

	count
duration_min	
112	7
95	6
13	5
79	5
16	4
...	...
82	1
115	1
93	1
94	1
26	1

95 rows × 1 columns

dtype: int64

```
df['duration_min'].value_counts().head(10)
```

	count
duration_min	
112	7
95	6
13	5
79	5
16	4
18	4
68	4
70	4
21	4
119	3

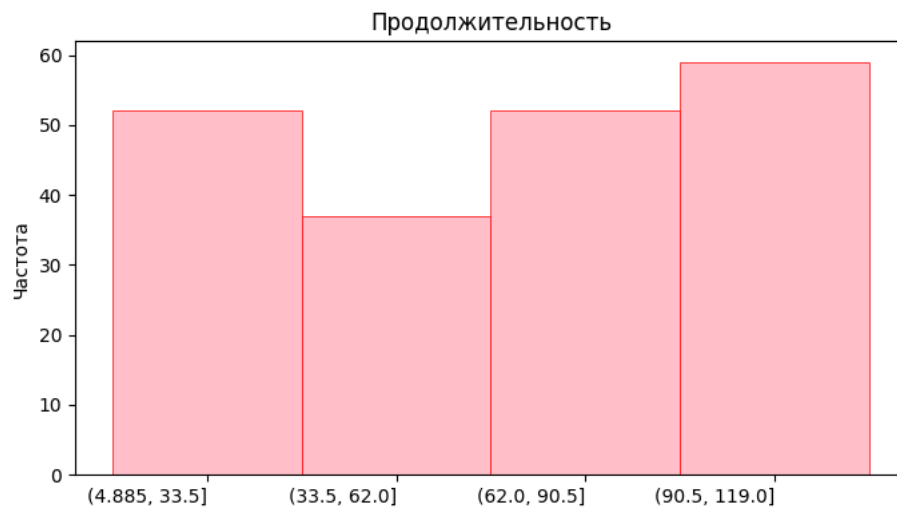
dtype: int64

```
df['duration_min'].value_counts(bins=4).sort_index()
```

	count
(4.885, 33.5]	52
(33.5, 62.0]	37
(62.0, 90.5]	52
(90.5, 119.0]	59

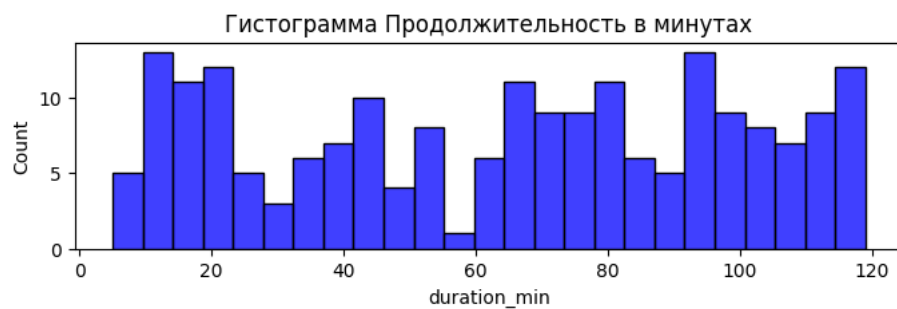
dtype: int64

```
intervals = ['(4.885, 33.5]', '(33.5, 62.0]', '(62.0, 90.5]', '(90.5, 119.0]']
frequencies = [52, 37, 52, 59]
fig, ax = plt.subplots(figsize=[7, 4])
x_pos = range(len(intervals))
plt.bar(x_pos, frequencies, color='pink', width=1.0, edgecolor='red', linewidth=0.5)
plt.xticks(x_pos, intervals, ha='right')
plt.title('Продолжительность')
plt.ylabel('Частота')
plt.tight_layout()
plt.show()
```



```
fig, ax = plt.subplots(figsize=[8, 2])
sns.histplot(data=df['duration_min'], bins=25, color='blue')
ax.set_title('Гистограмма Продолжительность в минутах', fontsize = 12)

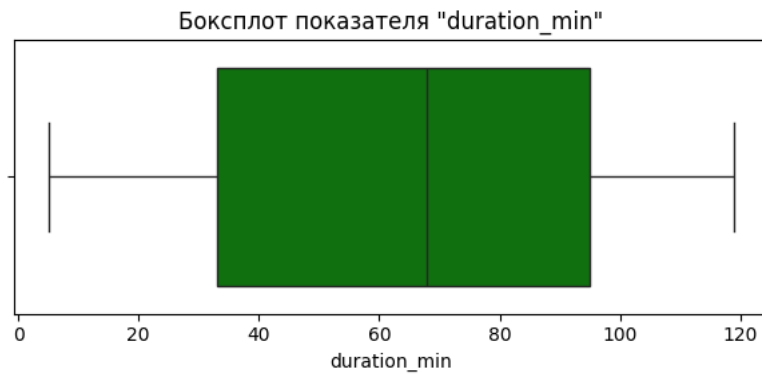
plt.show()
```



```
fig, ax = plt.subplots(1, 1, figsize=[6, 3])

sns.boxplot(data=df, x='duration_min', color='green', ax=ax)

plt.title('Боксплот показателя "duration_min"')
plt.tight_layout()
plt.show()
```



Выводы

Значения варьируются от 5 до 119

Среднее значение: 64.025

Медиана: 68 (50% пользователей имеют продолжительность 68 и ниже)

Первый квартиль: 33 (25% пользователей имеют продолжительность 33 и ниже)

Третий квартиль: 95 (75% пользователей имеют продолжительность 95 и ниже)

Стандартное отклонение от среднего значения: 34.69

Наибольшее количество пользователей имеют продолжительность от 95.5 до 119

3. calories_burned

```
df['calories_burned'].describe()
```

	calories_burned
count	200.000000
mean	295.765000
std	120.563834
min	50.000000
25%	220.000000
50%	311.500000
75%	378.000000
max	585.000000

dtype: float64

```
df['calories_burned'].value_counts()
```

```
count
calories_burned
50          10
308          3
376          3
281          3
313          3
...         ...
377          1
435          1
408          1
585          1
241          1
```

154 rows × 1 columns

dtype: int64

```
df['calories_burned'].value_counts().head(10)
```

```
count
calories_burned
50          10
308          3
376          3
281          3
313          3
369          3
169          3
318          3
230          3
330          3
```

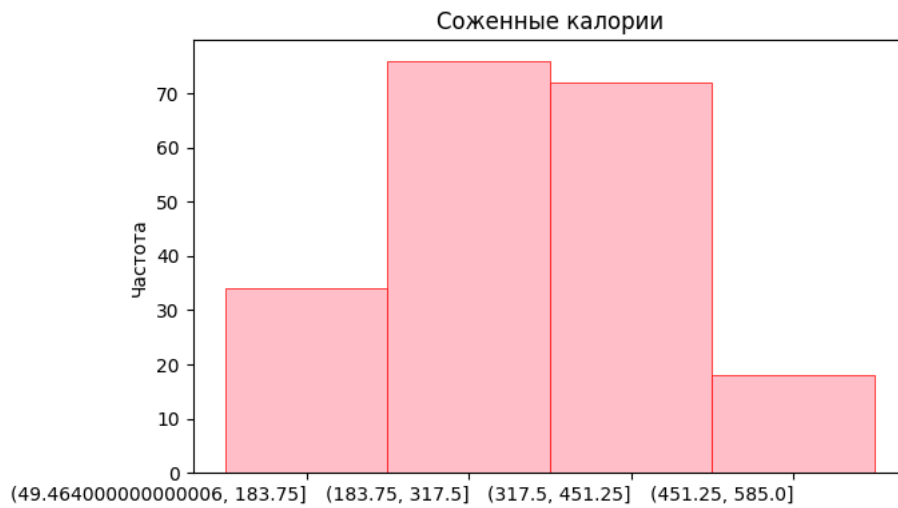
dtype: int64

```
df['calories_burned'].value_counts(bins=4).sort_index()
```

```
count
(49.464000000000006, 183.75]    34
(183.75, 317.5]                76
(317.5, 451.25]                72
(451.25, 585.0]                18
```

dtype: int64

```
intervals = ['(49.464000000000006, 183.75]', '(183.75, 317.5]', '(317.5, 451.25]', '(451.25, 585.0]']
frequencies = [34, 76, 72, 18]
fig, ax = plt.subplots(figsize=[7, 4])
x_pos = range(len(intervals))
plt.bar(x_pos, frequencies, color='pink', width=1.0, edgecolor='red', linewidth=0.5)
plt.xticks(x_pos, intervals, ha='right')
plt.title('Соженные калории')
plt.ylabel('Частота')
plt.tight_layout()
plt.show()
```

```
fig, ax = plt.subplots(figsize=[8, 2])
sns.histplot(data=df['calories_burned'], bins=25, color='blue')
ax.set_title('Гистограмма Сожженные калории', fontsize = 12)

plt.show()
```



```
fig, ax = plt.subplots(1, 1, figsize=[6, 3])

sns.boxplot(data=df, x='calories_burned', color='green', ax=ax)

plt.title('Боксплот показателя "calories_burned"')
plt.tight_layout()
plt.show()
```



Выводы

Значения варьируются от 50 до 585

Среднее значение: 295.765

Медиана: 311.5 (50% пользователей сжигают 311.5 калорий и меньше)

Первый квартиль: 220 (25% пользователей сжигают 220 калорий и меньше)

Третий квартиль: 378 (75% пользователей сжигают 378 калорий и меньше)

Стандартное отклонение от среднего значения: 120

Наибольшее количество пользователей сжигают калорий от 183.75 до 317.5

4. heart_rate_avg

```
df['heart_rate_avg'].describe()
```

	heart_rate_avg
count	200.000000
mean	120.935000
std	34.100366
min	60.000000
25%	89.750000
50%	121.000000
75%	150.250000
max	179.000000

dtype: float64

```
df['heart_rate_avg'].value_counts()
```

	count
heart_rate_avg	
163	6
102	6
85	6
165	5
179	4
...	...
116	1
135	1
125	1
86	1
72	1

95 rows × 1 columns

dtype: int64

```
df['heart_rate_avg'].value_counts().head(10)
```

	count
heart_rate_avg	
163	6
102	6
85	6
165	5
179	4
115	4
128	4
121	4
140	4
98	3

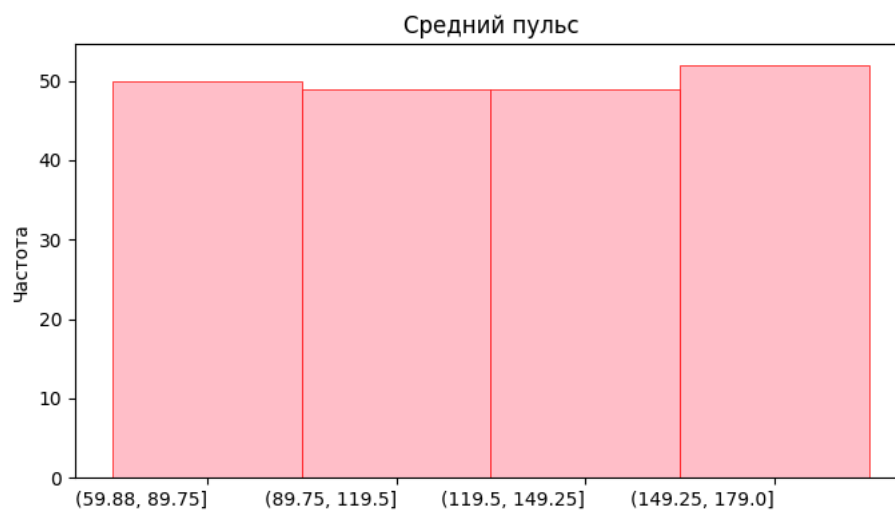
dtype: int64

```
df['heart_rate_avg'].value_counts(bins=4).sort_index()
```

	count
(59.88, 89.75]	50
(89.75, 119.5]	49
(119.5, 149.25]	49
(149.25, 179.0]	52

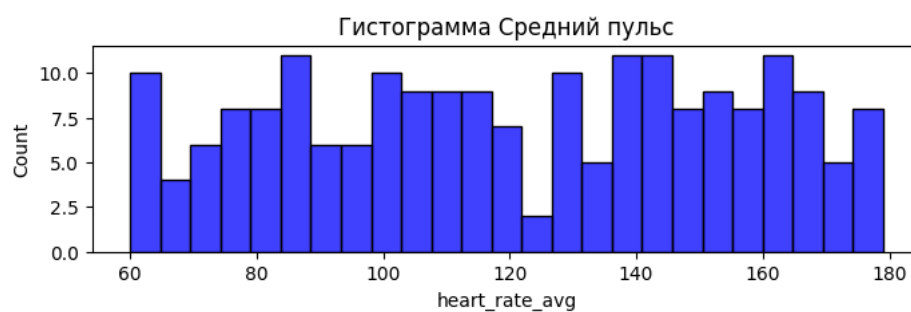
dtype: int64

```
intervals = ['(59.88, 89.75]', '(89.75, 119.5]', '(119.5, 149.25]', '(149.25, 179.0]']
frequencies = [50, 49, 49, 52]
fig, ax = plt.subplots(figsize=[7, 4])
x_pos = range(len(intervals))
plt.bar(x_pos, frequencies, color='pink', width=1.0, edgecolor='red', linewidth=0.5)
plt.xticks(x_pos, intervals, ha='right')
plt.title('Средний пульс')
plt.ylabel('Частота')
plt.tight_layout()
plt.show()
```



```
fig, ax = plt.subplots(figsize=[8, 2])
sns.histplot(data=df['heart_rate_avg'], bins=25, color='blue')
ax.set_title('Гистограмма Средний пульс', fontsize = 12)

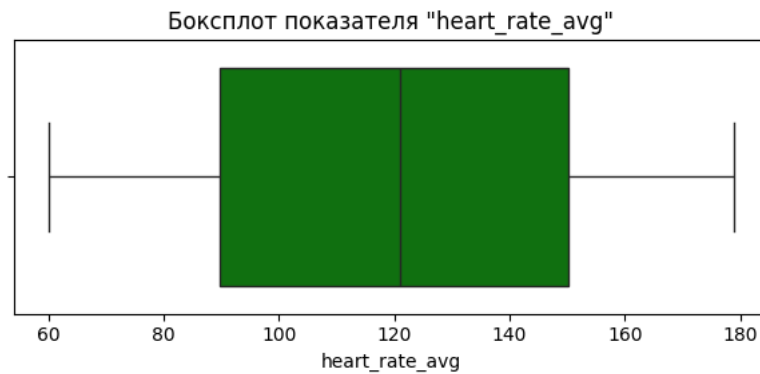
plt.show()
```



```
fig, ax = plt.subplots(1, 1, figsize=[6, 3])

sns.boxplot(data=df, x='heart_rate_avg', color='green', ax=ax)

plt.title('Боксплот показателя "heart_rate_avg"')
plt.tight_layout()
plt.show()
```



Выводы

Значения варьируются от 60 до 179

Среднее значение: 120.935

Медиана: 121 (50% пользователей имеют пульс 121 и ниже)

Первый квартиль: 89.75 (25% пользователей имеют пульс 89.75 и ниже)

Третий квартиль: 150.25 (75% пользователей имеют пульс 150.25 и ниже)

Стандартное отклонение от среднего значения: 34.100366

Наибольшее количество пользователей имеют частоту пульса от 59.88 до 89.75

5. steps

```
df['steps'].describe()
```

	steps
count	200.00000
mean	14020.82000
std	6428.05981
min	370.00000
25%	9585.00000
50%	14857.00000
75%	19007.75000
max	24984.00000

dtype: float64

```
df['steps'].value_counts().sort_index()
```

	count
steps	
370	1
633	1
721	1
960	1
1267	1
...	...
24435	1
24789	1
24819	1
24968	1
24984	1

200 rows × 1 columns

dtype: int64

df['steps'].value_counts().head(10)

	count
steps	
3591	1
22312	1
5418	1
19809	1
12562	1
23432	1
14851	1
15351	1
13072	1
22439	1

dtype: int64

df['steps'].value_counts(bins=4).sort_index()

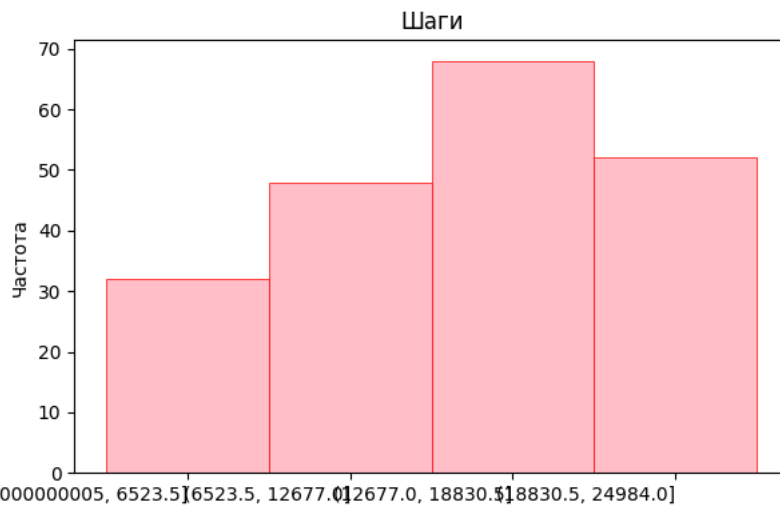
	count
(345.38500000000005, 6523.5]	32
(6523.5, 12677.0]	48
(12677.0, 18830.5]	68
(18830.5, 24984.0]	52

dtype: int64

```

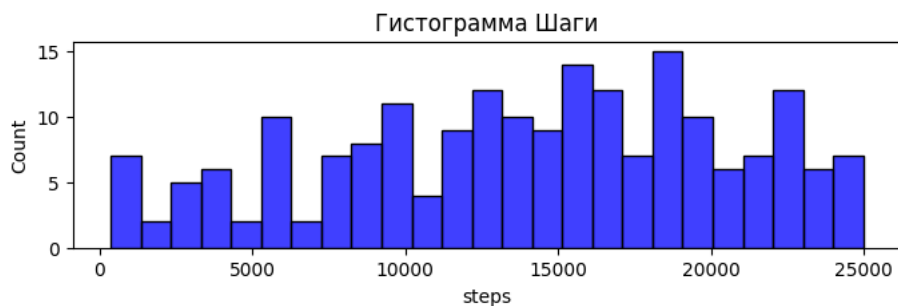
intervals = ['(345.38500000000005, 6523.5]', '(6523.5, 12677.0]', '(12677.0, 18830.5]', '(18830.5, 24984.0)']
frequencies = [32, 48, 68, 52]
fig, ax = plt.subplots(figsize=[7, 4])
x_pos = range(len(intervals))
plt.bar(x_pos, frequencies, color='pink', width=1.0, edgecolor='red', linewidth=0.5)
plt.xticks(x_pos, intervals, ha='right')
plt.title('Шаги')
plt.ylabel('Частота')
plt.tight_layout()
plt.show()

```



```
fig, ax = plt.subplots(figsize=[8, 2])
sns.histplot(data=df['steps'], bins=25, color='blue')
ax.set_title('Гистограмма Шаги', fontsize = 12)

plt.show()
```



```
fig, ax = plt.subplots(1, 1, figsize=[6, 3])

sns.boxplot(data=df, x='steps', color='green', ax=ax)

plt.title('Боксплот показателя "steps"')
plt.tight_layout()
plt.show()
```



Вывод

Значения варьируются от 370 до 24984

Среднее значение: 14020

Медиана: 14857 (50% пользователей ходят 14857 шагов и меньше)

Первый квартиль: 9585 (25% пользователей ходят 9585 шагов и меньше)

Третий квартиль: 19007 (75% пользователей ходят 19007 шагов и меньше)

Стандартное отклонение от среднего значения: 6428

Наибольшее количество пользователей ходит шагов от 12677.0 и до 18830.5

6. sleep_hours

```
df['sleep_hours'].describe()
```

	sleep_hours
count	200.000000
mean	6.313000
std	1.911518
min	3.000000
25%	4.775000
50%	6.500000
75%	7.900000
max	9.500000

dtype: float64

```
df['sleep_hours'].value_counts()
```

	count
sleep_hours	
6.8	8
9.4	7
4.2	7
3.3	6
7.4	5
...	...
7.1	1
4.0	1
5.3	1
8.3	1
8.0	1

63 rows × 1 columns

dtype: int64

```
df['sleep_hours'].value_counts().head(10)
```

	count
sleep_hours	
6.8	8
9.4	7
4.2	7
3.3	6
7.4	5
6.3	5
7.7	5
8.8	5
4.3	5
4.9	5

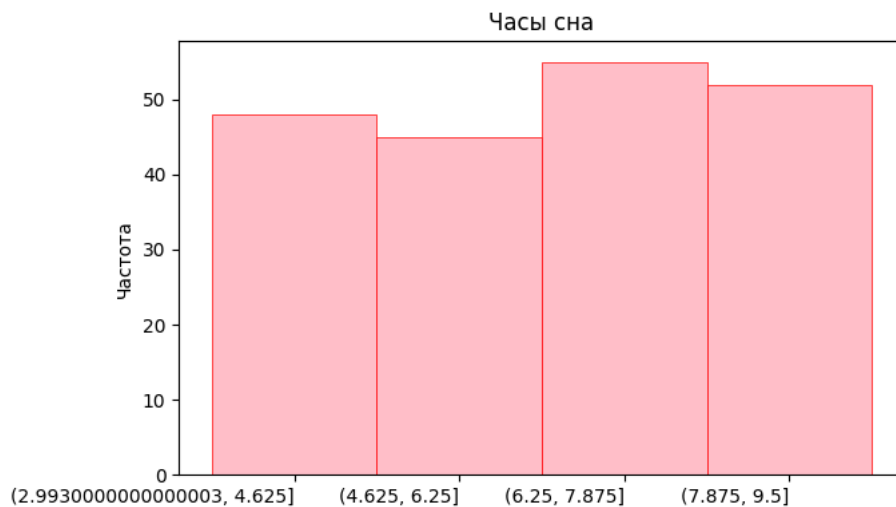
dtype: int64

```
df['sleep_hours'].value_counts(bins=4).sort_index()
```

	count
(2.9930000000000003, 4.625]	48
(4.625, 6.25]	45
(6.25, 7.875]	55
(7.875, 9.5]	52

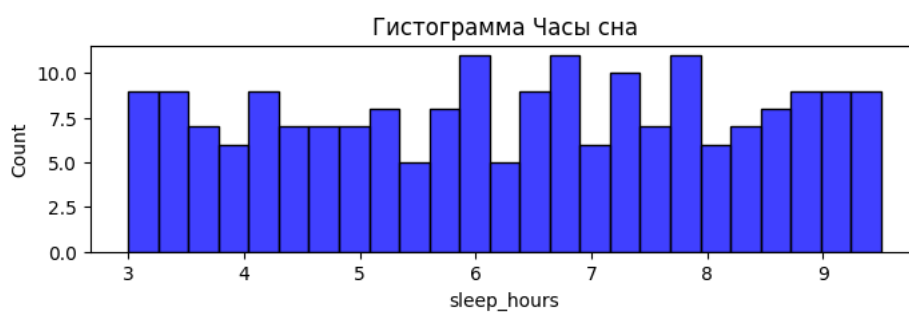
dtype: int64

```
intervals = ['(2.9930000000000003, 4.625]', '(4.625, 6.25]', '(6.25, 7.875]', '(7.875, 9.5]']
frequencies = [48, 45, 55, 52]
fig, ax = plt.subplots(figsize=[7, 4])
x_pos = range(len(intervals))
plt.bar(x_pos, frequencies, color='pink', width=1.0, edgecolor='red', linewidth=0.5)
plt.xticks(x_pos, intervals, ha='right')
plt.title('Часы сна')
plt.ylabel('Частота')
plt.tight_layout()
plt.show()
```



```
fig, ax = plt.subplots(figsize=[8, 2])
sns.histplot(data=df['sleep_hours'], bins=25, color='blue')
ax.set_title('Гистограмма Часы сна', fontsize = 12)

plt.show()
```

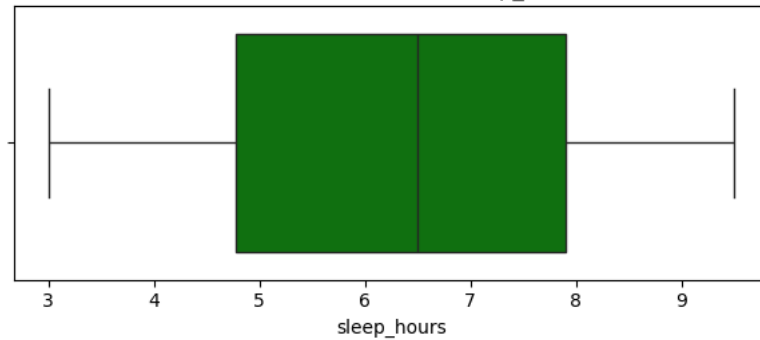


```
fig, ax = plt.subplots(1, 1, figsize=[6, 3])

sns.boxplot(data=df, x='sleep_hours', color='green', ax=ax)

plt.title('Боксплот показателя "sleep_hours"')
plt.tight_layout()
plt.show()
```


Боксплот показателя "sleep_hours"



Вывод

Значения варьируются от 3 до 9.5

Среднее значение: 6.313

Медиана: 6.5 (50% пользователей спят 6.5 часов и меньше)

Первый квартиль: 4.775 (25% пользователей спят 4.775 часов и меньше)

Третий квартиль: 7.9 (75% пользователей спят 7.9 часов и меньше)

Стандартное отклонение от среднего значения: 1.9

Наибольшее количество пользователей спят от 6.25 до 7.875 часов

7. weight_kg

```
df['weight_kg'].describe()
```

	weight_kg
count	200.000000
mean	70.152000
std	14.129562
min	35.300000
25%	61.450000
50%	70.600000
75%	80.100000
max	105.200000

dtype: float64

```
df['weight_kg'].value_counts()
```

	count
weight_kg	
61.8	3
73.3	3
82.7	3
58.0	2
72.0	2
...	...
71.2	1
46.2	1
89.4	1
98.6	1
72.1	1

171 rows × 1 columns

dtype: int64

df['weight_kg'].value_counts().head(10)

	count
weight_kg	
61.8	3
73.3	3
82.7	3
58.0	2
72.0	2
56.4	2
73.4	2
55.0	2
55.4	2
69.4	2

dtype: int64

df['weight_kg'].value_counts(bins=4).sort_index()

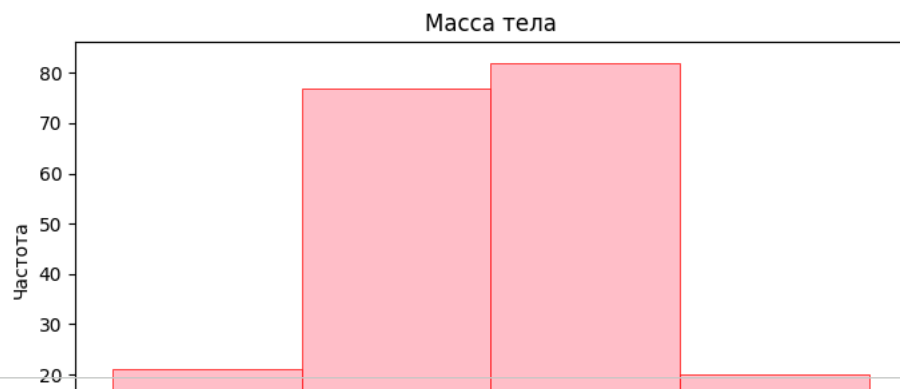
	count
(35.229, 52.775]	21
(52.775, 70.25]	77
(70.25, 87.725]	82
(87.725, 105.2]	20

dtype: int64

```

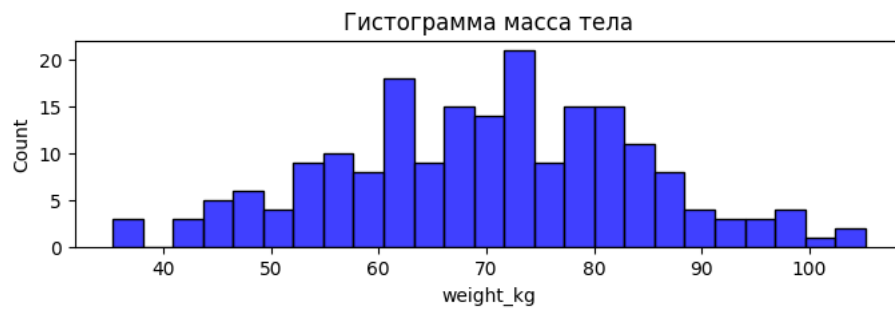
intervals = ['(35.229, 52.775]', '(52.775, 70.25]', '(70.25, 87.725]', '(87.725, 105.2]']
frequencies = [21, 77, 82, 20]
fig, ax = plt.subplots(figsize=[7, 4])
x_pos = range(len(intervals))
plt.bar(x_pos, frequencies, color='pink', width=1.0, edgecolor='red', linewidth=0.5)
plt.xticks(x_pos, intervals, ha='right')
plt.title('Macca тена')
plt.ylabel('Частота')
plt.tight_layout()
plt.show()

```



```
fig, ax = plt.subplots(figsize=[8, 2])
sns.histplot(data=df['weight_kg'], bins=25, color='blue')
ax.set_title('Гистограмма масса тела', fontsize = 12)

plt.show()
```



```
fig, ax = plt.subplots(1, 1, figsize=[6, 3])

sns.boxplot(data=df, x='weight_kg', color='green', ax=ax)

plt.title('Боксплот показателя "weight_kg"')
plt.tight_layout()
plt.show()
```